

UNIVERSIDAD DE SONORA



DESARROLLO DE SISTEMAS III

Carrera: Ingeniería en sistemas de información

Profesor: Lev David Valenzuela López

Proyecto final de la materia de Desarrollo de Software III

Integrantes:

David Antonio López Corella

Perla Jazmín Marquez Martinez

Brandon Isaac Miranda Montes

Natalia Guadalupe Sánchez Valenzuela

Daniela Sandoval Lopez

Fei Fei Wu Zhang

03/12/25

Hermosillo, Son.

Descripción General:

Como proyecto final, desarrollamos un Sistema de *Monitoreo de Interfaces de Red*, una aplicación de escritorio en C# con Windows Forms cuyo propósito es capturar el estado actual de todas las interfaces de red del equipo y guardar ese registro de forma segura en una base de datos SQL Server. Para manejar de manera eficiente las operaciones que consumen tiempo, como el escaneo de red y la comunicación con la base de datos, el sistema utiliza la programación asíncrona (*async/await*) con hilos de sistema, lo que es importante para asegurar que el formulario principal nunca se congele, manteniendo la interfaz de usuario fluida y funcional.

Base de datos:

Esta base de datos es la memoria del Sistema de Monitoreo, la diseñamos para almacenar de forma eficiente los resultados capturados por los procesos asíncronos de la aplicación. La estructura organiza la información en dos niveles lógicos: la tabla lote, que registra la marca de tiempo exacta de cada ejecución, y la tabla interfaz, que almacena los detalles técnicos profundos como IP, MAC y estado operativo, lo que permite así reconstruir la configuración de red del equipo en cualquier instante que haya sido capturado.

```
-- 1. Tabla de Lotes de Ejecución (ExecutionBatch)
-- Esta tabla registra cada vez que el script de C# es ejecutado.
CREATE TABLE lote (
    -- Clave primaria que identifica de forma única cada ejecución del script
    ID_lote INT PRIMARY KEY IDENTITY(1,1),

    -- Columna para registrar la fecha y hora de la captura del lote
    ExecutionTime DATETIME2(0) NOT NULL DEFAULT GETDATE()
);
GO
```

```

-- 2. Tabla de Interfaces de Red (NetworkInterface)
-- Se relaciona con el lote mediante el RunID.
CREATE TABLE interfaz (
    -- Clave primaria, auto-incrementable
    id_interface INT PRIMARY KEY IDENTITY(1,1),

    -- Clave foránea que enlaza esta interfaz con un lote de ejecución específico
    id_lote INT NOT NULL,

    -- Nombre de la interfaz
    nombre NVARCHAR(256) NOT NULL,

    -- Descripción del adaptador de red
    descripcion NVARCHAR(256) NOT NULL,

    -- Tipo de interfaz (ej. 'Ethernet', 'Wireless80211', 'Loopback')
    tipo_interfaz NVARCHAR(50) NOT NULL,

    -- Estado operativo (ej. 'Up', 'Down')
    estatus NVARCHAR(50) NOT NULL,

    -- Dirección MAC, siempre en formato 00:00:00:00:00:00
    MacAddress CHAR(17) NULL,

    -- Dirección IPv4. Se acepta NULL si no hay una dirección IPv4 asignada.
    IPv4Address NVARCHAR(45) NULL,

    -- Dirección IPv6. Se acepta NULL si no hay una dirección IPv6 asignada.
    IPv6Address NVARCHAR(45) NULL,

    -- Definición de la clave foránea que asegura la agrupación por lote
    CONSTRAINT FK_Batch_Interface
        FOREIGN KEY (id_lote)
        REFERENCES lote(id_lote)
        -- Si un lote (ExecutionBatch) se borra, también se borran todas las interfaces asociadas.
        ON DELETE CASCADE
);
GO

```

Form Principal:

Maneja la interfaz de usuario, permitiendo al usuario escanear las interfaces de red de su equipo, almacenando estos resultados temporalmente en la lista *_lastScanResults*. Posteriormente, el usuario puede presionar "Guardar" para mantener esta información en una base de datos SQL Server utilizando la base de datos *NetworkDB*, donde los datos se organizan por Lotes (*dbo.lote*) y sus correspondientes detalles de interfaces (*dbo.interfaz*), un proceso gestionado por la clase *DatabaseManager* en hilos asíncronos. Al iniciar la aplicación, la cuadrícula principal muestra el historial de todos los lotes guardados (*LoadHistory*), y el usuario puede hacer clic en cualquier fila del historial para cargar y visualizar los detalles de las interfaces correspondientes a ese lote específico (*LoadInterfaceDetails*).

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Drawing;
using System.Threading.Tasks;
using System.Windows.Forms;
using static System.Windows.Forms.VisualStyles.VisualStyleElement.ListView;

namespace ProyectoDesa3
{
    4 referencias
    public partial class formsprincipal : Form
    {
        private readonly DatabaseManager dbManager = new DatabaseManager();
        private List<InterfazData> _lastScanResults = new List<InterfazData>();

        1 referencia
        public formsprincipal()
        {
            InitializeComponent();
            ConfigurarDataGridView();
            LoadHistory(); // Carga el historial al iniciar
        }

        1 referencia
        private void ConfigurarDataGridView()
        {
            dgvDatos.AutoGenerateColumns = true;
            dgvDatos.ReadOnly = true;
            dgvDatos.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
            dgvDatos.AllowUserToAddRows = false;

            dgvDatos.CellClick += new DataGridViewCellEventHandler(dgvDatos_CellContentClick);
        }
    }
}
```

4 referencias

```
private async void LoadHistory()
{
    try
    {
        DataTable history = await dbManager.GetLoteHistoryAsync();
        if (dgvDatos.IsHandleCreated)
        {
            dgvDatos.BeginInvoke(new MethodInvoker(() =>
            {
                dgvDatos.DataSource = history;
                // Renombrar encabezados del historial
                if (dgvDatos.Columns.Contains("ID_lote"))
                {
                    dgvDatos.Columns["ID_lote"].HeaderText = "ID Lote";
                }
                if (dgvDatos.Columns.Contains("ExecutionTime"))
                {
                    dgvDatos.Columns["ExecutionTime"].HeaderText = "Fecha y Hora de Escaneo";
                }

                dgvDatos.Refresh();
            }));
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Error al cargar el historial: {ex.Message}", "Error de BD", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

2 referencias

```
private async void dgvDatos_CellContentClick(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex < 0) return;

    if (dgvDatos.Columns.Contains("ExecutionTime") && dgvDatos.Rows.Count > e.RowIndex)
    {
        try
        {
            int loteId = Convert.ToInt32(dgvDatos.Rows[e.RowIndex].Cells["ID_lote"].Value);

            await LoadInterfaceDetails(loteId);
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Error al seleccionar lote: {ex.Message}", "Error de Selección", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        }
    }
}
```

1 referencia

```
private async void tsbEscanear_Click_1(object sender, EventArgs e)
{
    tsbEscanear.Enabled = false;
    Cursor = Cursors.WaitCursor;
    DateTime horaInicio = DateTime.Now;

    // 1. Ejecutar Escaneo
    _lastScanResults = null; // Limpiar resultados anteriores
    try
    {
        _lastScanResults = await Task.Run(() =>
        {
            return ScannerInterfaces.EjecutarEscaneo();
        });
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Error durante el escaneo de interfaces: {ex.Message}", "Error de Escaneo", MessageBoxButtons.OK, MessageBoxIcon.Error);
        tsbEscanear.Enabled = true;
        Cursor = Cursors.Default;
        return;
    }

    // 2. MUESTRA LOS RESULTADOS DEL ESCANEO DIRECTAMENTE
    string mensaje = $"Escaneeo completado. Interfaces encontradas: {_lastScanResults.Count}.";

    if (dgvDatos.IsHandleCreated)
    {
        dgvDatos.BeginInvoke(new MethodInvoker(() =>
        {
            // Muestra los resultados en el DGV
            dgvDatos.DataSource = _lastScanResults;
            dgvDatos.Refresh();
        }));
    }

    // 3. MENSAJE FINAL
    DateTime horaFin = DateTime.Now;
    TimeSpan duracion = horaFin - horaInicio;
    mensaje += "\nListo para guardar en la base de datos. Presione 'Guardar'.";

    MessageBox.Show($"{mensaje}\nDuración total: {duracion.TotalSeconds:F2} segundos.", "Escaneeo Completo", MessageBoxButtons.OK, MessageBoxIcon.Information);

    // 4. Desbloquear UI
    tsbEscanear.Enabled = true;
    Cursor = Cursors.Default;
}
```

```

1 referencia
private void btnVerHistorial_Click(object sender, EventArgs e)
{
    LoadHistory();
}

1 referencia
private async void tsbGuardar_Click(object sender, EventArgs e)
{
    if (_lastScanResults == null || _lastScanResults.Count == 0)
    {
        MessageBox.Show("No hay resultados de escaneo para guardar. Ejecute 'Escanear' primero.", "Advertencia", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    tsbGuardar.Enabled = false;
    Cursor = Cursors.WaitCursor;
    int loteId = -1;

    try
    {
        // LLAMADA AL GUARDADO EN LA BASE DE DATOS
        loteId = await dbManager.SaveScanResultsAsync(_lastScanResults);
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Error al guardar resultados en la base de datos. Mensaje de SQL: {ex.Message}", "Error Critico de Base de Datos", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    finally
    {
        tsbGuardar.Enabled = true;
        Cursor = Cursors.Default;
    }

    if (loteId > 0)
    {
        MessageBox.Show($"Datos guardados correctamente en el Lote ID: {loteId}.", "Guardado Exitoso", MessageBoxButtons.OK, MessageBoxIcon.Information);

        LoadHistory();
    }
    else
    {
        MessageBox.Show("El proceso de guardado falló. Verifique la conexión a la base de datos.", "Error de Guardado", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
}

```

InterfazData:

En esta clase se define la estructura de datos *InterfazData* para descubrir las interfaces de red en el sistema, mostrando la información de cada adaptador. La clase estática *ScannerInterfaces* contiene el método *EjecutarEscaneo()*, el cual interactúa con el sistema operativo utilizando *System.Net.NetworkInformation* para obtener una lista completa de adaptadores (*NetworkInterface.GetAllNetworkInterfaces()*). El método itera sobre cada adaptador, recopila su dirección MAC y busca las primeras direcciones *IP unicast*, capturando el estado operativo para finalmente devolver una lista de objetos *InterfazData* con la información detallada de la configuración de red del equipo.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Net.NetworkInformation;
using System.Net.Sockets;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;

namespace ProyectoDesa3
{
    8 referencias
    public class InterfazData
    {
        2 referencias
        public string Descripcion { get; set; }
        2 referencias
        public string Nombre { get; set; }
        2 referencias
        public string Tipo { get; set; }
        2 referencias
        public string Estado { get; set; }
        3 referencias
        public string MacAddress { get; set; }
        4 referencias
        public string IPv4 { get; set; }
        3 referencias
        public bool TieneDireccionIP { get; set; }
    }
    1 referencia
```

```

1 referencia
public static class ScannerInterfaces
{
    1 referencia
    public static List<InterfazData> EjecutarEscaneo()
    {
        List<InterfazData> resultados = new List<InterfazData>();

        NetworkInterface[] adaptadores = NetworkInterface.GetAllNetworkInterfaces();

        foreach (NetworkInterface adaptador in adaptadores)
        {
            InterfazData data = new InterfazData
            {
                Descripcion = adaptador.Description,
                Nombre = adaptador.Name,
                Tipo = adaptador.NetworkInterfaceType.ToString(),
                Estado = adaptador.OperationalStatus == OperationalStatus.Up ? "Activa" : "Inactiva",
                MacAddress = "No disponible",
                IPv4 = "N/A",
                IPv6 = "N/A",
                TieneDireccionIP = false
            };

            // 1. Dirección MAC
            string macAddress = adaptador.GetPhysicalAddress().ToString();
            if (!string.IsNullOrEmpty(macAddress) && macAddress != "000000000000")
            {
                // Formatea la dirección MAC (XX:XX:XX:XX:XX:XX)
                data.MacAddress = Regex.Replace(macAddress, ".{2}", "$0:").TrimEnd(':');
            }
        }
    }
}

```

```

        // 2. Direcciones IP
        try
        {
            IPInterfaceProperties propiedadesIP = adaptador.GetIPProperties();

            foreach (UnicastIPAddressInformation ip in propiedadesIP.UnicastAddresses)
            {
                if (ip.Address.AddressFamily == AddressFamily.InterNetwork && data.IPv4 == "N/A")
                {
                    data.IPv4 = ip.Address.ToString();
                    data.TieneDireccionIP = true;
                }
                else if (ip.Address.AddressFamily == AddressFamily.InterNetworkV6 && data.IPv6 == "N/A")
                {
                    data.IPv6 = ip.Address.ToString();
                    data.TieneDireccionIP = true;
                }
            }
        }
        catch (NetworkInformationException)
        {
            // Manejo de errores de obtención de IP
        }

        resultados.Add(data);
    }

    return resultados;
}

```

DatabaseManager:

Maneja de manera asíncrona la comunicación con la base de datos SQL Server, aislando las operaciones de *Entrada/Salida* del hilo principal de la aplicación mediante *Task.Run*. Su función crítica, *SaveScanResultsAsync*, ejecuta una transacción SQL para asegurar la integridad de los datos: primero inserta un registro en la tabla *dbo.lote* para obtener un ID, y luego inserta los múltiples resultados de las interfaces de red en la tabla *dbo.interfaz*, realizando un rollback si cualquier paso falla.

Adicionalmente, provee métodos asíncronos para la consulta de historial (*GetLoteHistoryAsync*), que recupera todos los lotes de escaneo, y la consulta de detalles (*GetInterfaceDetailsAsync*), que extrae los datos de interfaz asociados a un ID de lote específico.

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Data.SqlClient;
using System.Threading.Tasks;
using Microsoft.Data.SqlClient;

namespace ProyectoDesa3
{
    2 referencias
    public class DatabaseManager
    {
        private const string ConnectionString = "Server=DANIFLOW\\SQLEXPRESS;"
                                                + " Database=NetworkDB;"
                                                + " Integrated Security=True;"
                                                + " TrustServerCertificate=True";
    }
}
```

```
1 referencia
public async Task<int> SaveScanResultsAsync(List<InterfazData> interfaces)
{
    int loteId = -1;
    Exception dbException = null;
    await Task.Run(() =>
    {
        using (SqlConnection connection = new SqlConnection(ConnectionString))
        {
            SqlTransaction transaction = null;

            try
            {
                connection.Open();
                transaction = connection.BeginTransaction();

                string insertLoteSql = "INSERT INTO dbo.lote (ExecutionTime) VALUES (@FechaHora); SELECT SCOPE_IDENTITY();";
                using (SqlCommand commandLote = new SqlCommand(insertLoteSql, connection, transaction))
                {
                    commandLote.Parameters.AddWithValue("@FechaHora", DateTime.Now);
                    loteId = Convert.ToInt32(commandLote.ExecuteScalar());
                }

                string insertInterfazSql = @"
                INSERT INTO dbo.interfaz (id_lote, nombre, descripcion, tipo_interfaz, estatus, MacAddress, IPv4Address, IPv6Address)
                VALUES (@id_lote, @nombre, @descripcion, @tipo_interfaz, @estatus, @MacAddress, @IPv4Address, @IPv6Address)";

                foreach (var interfaz in interfaces)
                {
                    using (SqlCommand commandInterfaz = new SqlCommand(insertInterfazSql, connection, transaction))
                    {
                        commandInterfaz.Parameters.AddWithValue("@id_lote", loteId);
                        commandInterfaz.Parameters.AddWithValue("@nombre", (object)interfaz.Nombre ?? DBNull.Value);
                        commandInterfaz.Parameters.AddWithValue("@descripcion", (object)interfaz.Descripcion ?? DBNull.Value);
                        commandInterfaz.Parameters.AddWithValue("@tipo_interfaz", (object)interfaz.Tipo ?? DBNull.Value);
                        commandInterfaz.Parameters.AddWithValue("@estatus", (object)interfaz.Estado ?? DBNull.Value);
                        commandInterfaz.Parameters.AddWithValue("@MacAddress", (object)interfaz.MacAddress ?? DBNull.Value);
                        commandInterfaz.Parameters.AddWithValue("@IPv4Address", (object)interfaz.IPv4 ?? DBNull.Value);
                        commandInterfaz.Parameters.AddWithValue("@IPv6Address", (object)interfaz.IPv6 ?? DBNull.Value);

                        commandInterfaz.ExecuteNonQuery();
                    }
                }

                transaction.Commit();
            }
            catch { }
        }
    });
}
```

```

        }
        catch (Exception ex)
        {
            dbException = ex;
            if (transaction != null)
            {
                transaction.Rollback();
            }
            loteId = -1;
        }
    }
});

if (dbException != null)
{
    System.Diagnostics.Debug.WriteLine($"Error crítico de BD en SaveScanResultsAsync: {dbException.Message}");
    throw dbException;
}

return loteId;
}

1 referencia
public async Task<DataTable> GetLoteHistoryAsync()
{
    DataTable dt = new DataTable();

    await Task.Run(() =>
    {
        using (SqlConnection connection = new SqlConnection(ConnectionString))
        {
            string selectSql = "SELECT ID_lote, ExecutionTime FROM dbo.lote ORDER BY ExecutionTime DESC";

            try
            {
                connection.Open();
                using (SqlCommand command = new SqlCommand(selectSql, connection))
                using (SqlDataAdapter adapter = new SqlDataAdapter(command))
                {
                    adapter.Fill(dt);
                }
            }
            catch (Exception ex)
            {
                System.Diagnostics.Debug.WriteLine($"Error al cargar el historial de lotes: {ex.Message}");
            }
        }
    });

    return dt;
}

```